
🧠 Kubernetes Control Plane (Expanded)

1. 🛟 kube-apiserver

Purpose:

Acts as the **front-end** for the Kubernetes control plane. It handles **RESTful requests** from users, CLI tools (like **kubectl**), and other control plane components.

Key Responsibilities:

- Validates requests (syntax, schema).
- Authenticates and authorizes.
- Persists the desired state to **etcd**.
- Communicates with other control plane components.

Example:

```
kubectl create deployment nginx --image=nginx
```

This command hits the **kube-apiserver**, which:

- Validates the request.
 - Authenticates you.
 - Updates the desired state in **etcd**.
 - Triggers relevant controllers (e.g., **Deployment Controller**).
-

2. etcd

Purpose:

A **highly available key-value store** used as the **backing store** for all cluster data. Every configuration, node, Pod, Secret, etc., lives here.

Example Use:

- If the cluster crashes and restarts, Kubernetes rebuilds the state from **etcd**.

Example Content:

/keynamespaces/default

/keypods/nginx-123abc

Command (backup):

```
ETCDCTL_API=3 etcdctl snapshot save snapshot.db
```

3. kube-scheduler

Purpose:

Selects a **suitable Node** for a newly created Pod based on:

- Resource availability
- Node affinity/taints
- Constraints and policies

How It Works:

- Watch unassigned Pods from the API server.
- Scores all nodes based on constraints.

- Binds the Pod to a node.

Example:

If a new Pod needs 2 CPUs and 4GB memory, and only **node-3** can handle it, the **kube-scheduler** binds the Pod there.

4. **kube-controller-manager**

Purpose:

Runs various **controller loops**, each responsible for continuously regulating the **desired vs. current state**.

Major Controllers:

- **Node Controller** – Monitors node health.
- **Replication Controller** – Ensures correct # of Pods.
- **Job Controller** – Manages Jobs and completion.
- **Endpoint Controller** – Populates **Endpoints** objects.

Example:

If a Deployment expects 3 Pods, but only 2 are running, the Replication Controller spins up 1 more Pod.

5. **cloud-controller-manager**

Purpose:

Abstracts and integrates Kubernetes with **cloud provider APIs**.

Main Functions:

- Attach/detach cloud volumes.

- Manage cloud-based load balancers.
- Handle node lifecycle (e.g., detect terminated VM).

Example Providers:

- AWS, GCP, Azure

Use Case:

When a node VM is deleted in AWS, the cloud controller manager detects this and removes the node from the cluster.

6. 🧩 Custom Resource Definitions (CRD)

Purpose:

Allows users to define **custom resources** that extend the Kubernetes API **without modifying the source code**.

Example:

You define a new type called **CronTab**:

```
apiVersion: "stable.example.com/v1"
kind: CronTab
metadata:
  name: my-new-cron-object
spec:
  cronSpec: "* * * * */5"
  image: my-cron-image
```

Now you can manage **CronTab** resources just like native Kubernetes objects.

7. 🗝️ Admission Controllers

Purpose:

Pluggable modules that **intercept API requests** after authentication/authorization but **before persistence in etcd**.

Two Types:

- **Mutating:** Modify requests (e.g., inject sidecars).
- **Validating:** Reject bad configurations.

Example:

- The **ResourceQuota** controller ensures a namespace doesn't exceed its limits.
 - The **PodSecurityPolicy** might deny a Pod that tries to run as root.
-

8. API Aggregation Layer

Purpose:

Allows Kubernetes to **expose external APIs** through the Kubernetes API server, grouped under `/apis`.

Use Case:

Let's say you want to expose a metrics API (like `metrics.k8s.io`) that isn't built into Kubernetes.

Example:

The **metrics-server** adds the `/apis/metrics.k8s.io/v1beta1` endpoint via aggregation.

Summary Table

Component	Role	Example
kube-apiserver	Central API gateway	<code>kubectl create pod</code> → triggers API validation and persistence

etcd	Persistent key-value store	Cluster state backed up via etcdctl snapshot
kube-scheduler	Assigns Pods to Nodes	Schedules Pod with 2 CPUs to a node that meets criteria
kube-controller-manager	Runs controllers to reconcile state	Ensures 3 replicas are always running
cloud-controller-manager	Integrates with cloud providers	Removes terminated cloud VMs from the cluster
CRD	Adds new custom APIs	Define a custom CronTab resource
Admission Controllers	Validates or mutates API requests	Deny privileged Pod via security policy
API Aggregation Layer	Adds 3rd-party APIs under /apis	/apis/metrics.k8s.io for metrics-server
